

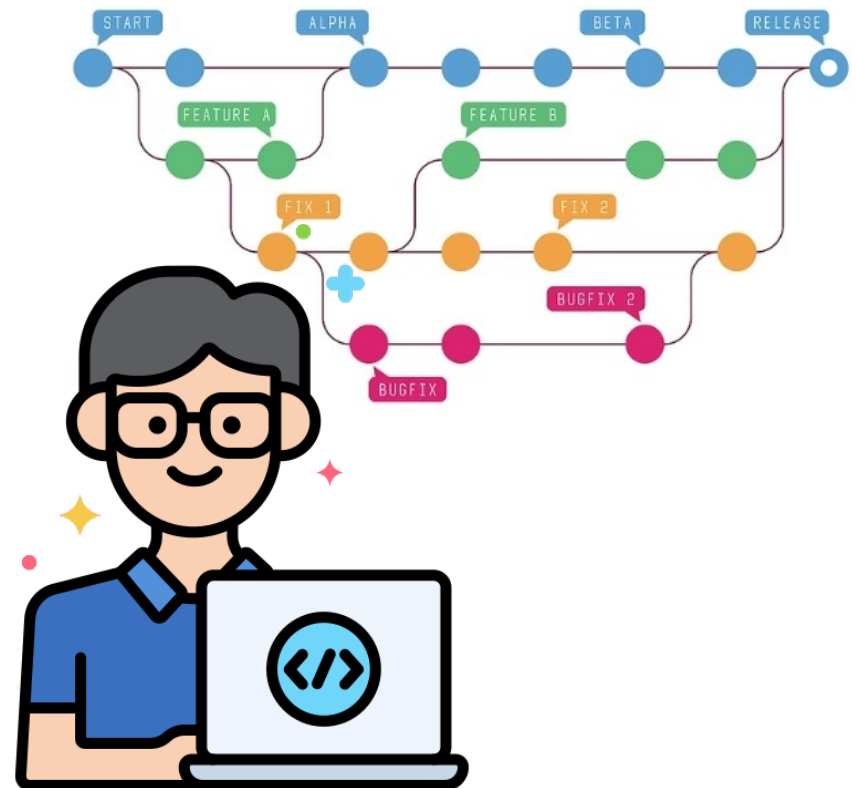
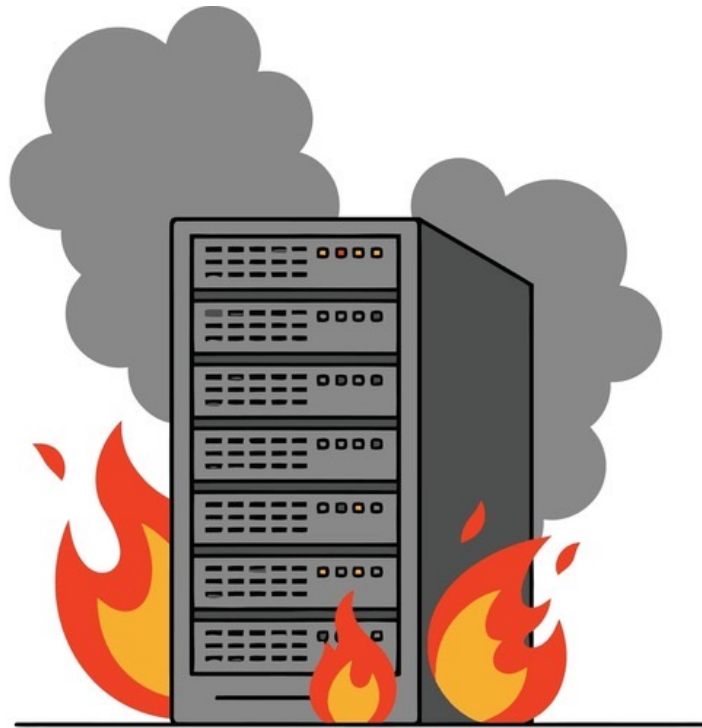


Gestão da Configuração e Versionamento

CÁSSIO CAPUCHO PEÇANHA

Gestão de Configuração e DevOps

Como transformar código em produto sem perder a sanidade.



A Esteira de Fábrica de Software

MATÉRIA-PRIMA:

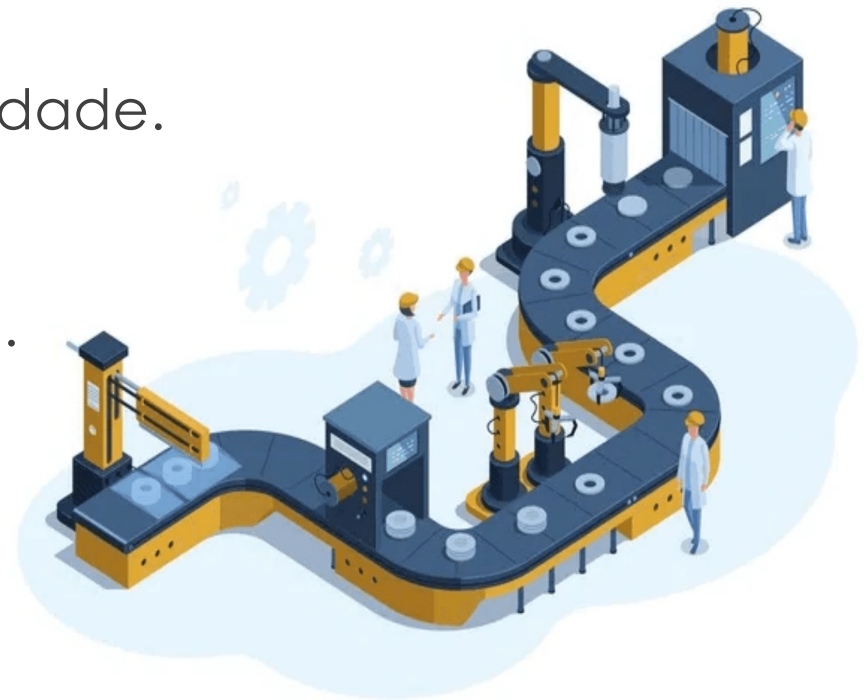
- O código fonte enviado pelos desenvolvedores.

ESTAÇÕES DE TRABALHO:

- Build, Testes e Análise de Qualidade.

PRODUTO FINAL:

- Software pronto para o cliente.



"Por que estamos aqui?"

- "Funciona na minha máquina" (A frase proibida).
- Pastas como TrabalhoFinal_v2_AGORA_VAI.zip.
- Medo de fazer deploy na sexta-feira.
 - Deploys manuais são lentos e propensos a erros humanos.
- Perder código porque o HD queimou ou sobrescreveu o arquivo do colega.
- Gargalos na integração geram atrasos e retrabalho constante.



A disciplina de colocar ordem na casa

O Conceito de SCM (Software Configuration Management)

Definição Simples:

➤ SCM é a arte de controlar mudanças no software.

Os 4 Pilares do SCM:

- Identificação: O que temos? (Código, Imagens, Banco).
- Controle: Quem mexeu? Quando? Por quê? (Git).
- Auditoria: O que foi entregue bate com o que foi pedido?
- Relatório: Qual o estado atual do projeto?

Engenharia de Software é... Engenharia.

O Código Fonte = A Matéria Prima (Aço/Borracha).

O Repositório (Git) = O Estoque Organizado.

O CI (Build/Testes) = O Controle de Qualidade Automatizado (Robôs testando se a porta fecha).

O CD (Deploy) = O Caminhão levando o carro para a concessionária (Cliente).

Continuous Integration & Continuous Delivery

CI (INTEGRAÇÃO CONTÍNUA):

- Mesclar código todo dia (nada de ficar 1 mês com código parado na máquina).
- Testes automáticos a cada save.

CD (ENTREGA CONTÍNUA):

- Se o teste passou, o software vai para o ar sozinho.
- Sem intervenção manual (sem FTP, sem arrastar pastinha).

O que vamos usar?

Tudo na Nuvem!

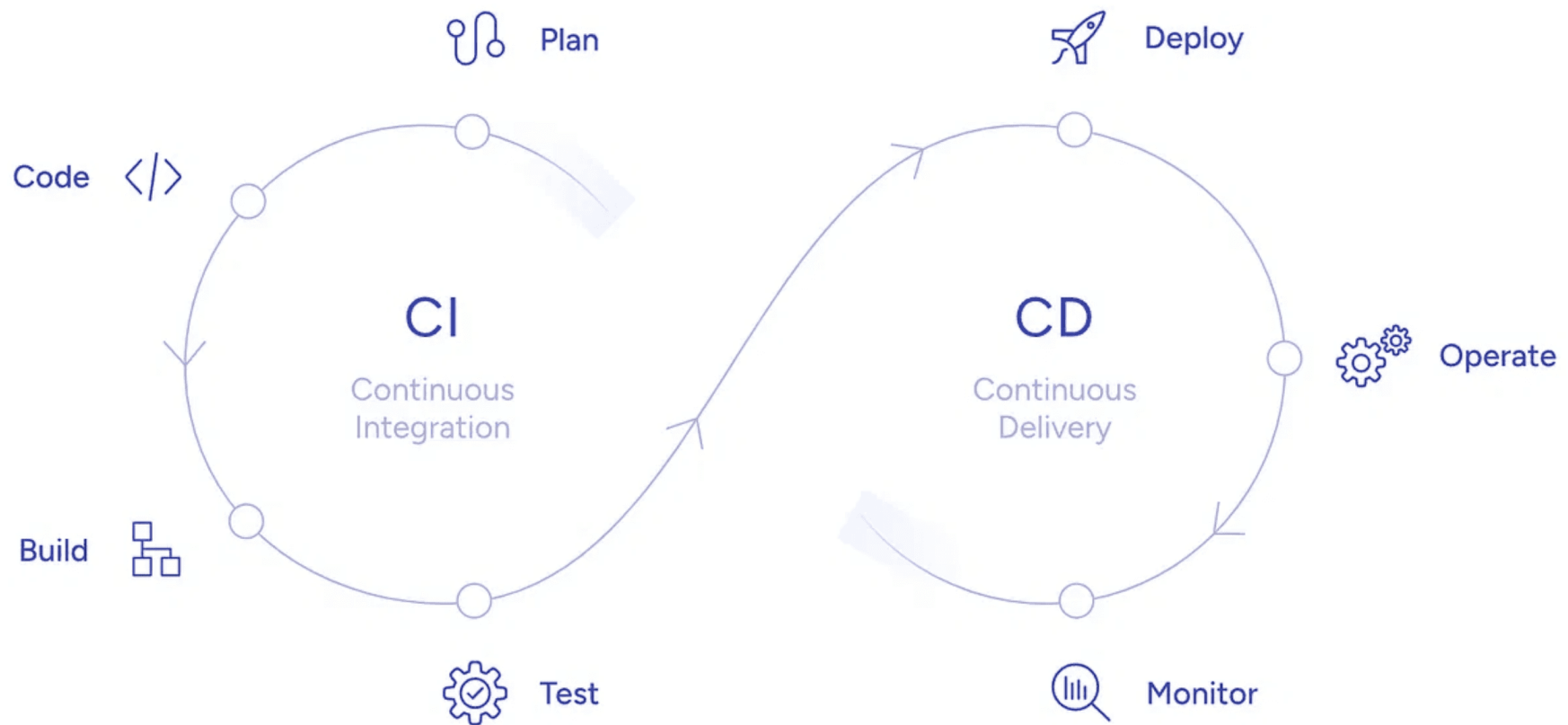
Git: A ferramenta de versão (o motor).

GitHub: Onde o código mora (a rede social).

GitHub Actions: O nosso "Robô" de automação (CI).

Vercel: Onde o site vai rodar (CD).

VS Code (Web): Nosso ambiente de desenvolvimento.

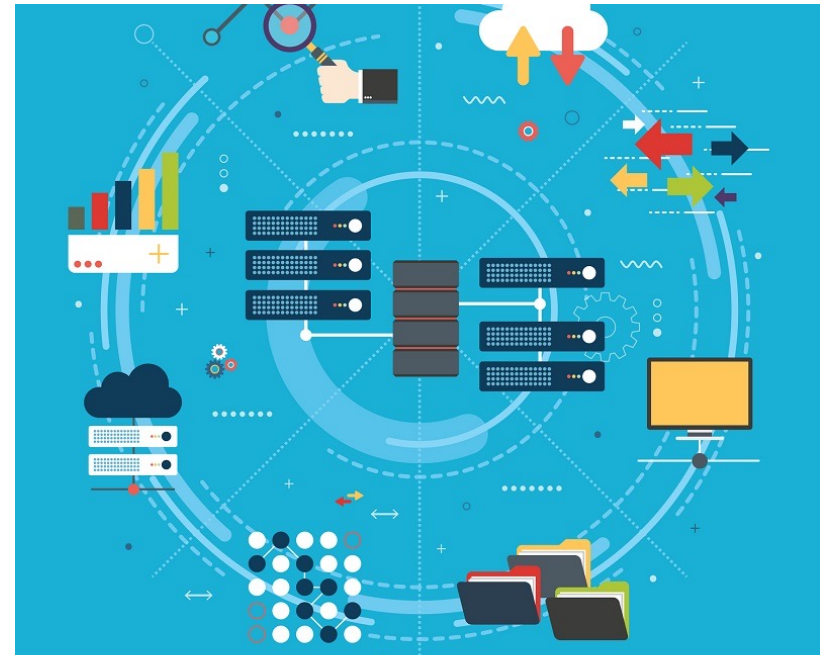


DevOps

[...] existe uma tendência em ver DevOps como um movimento que visa introduzir práticas ágeis na “última milha” de um projeto de software, isto é, quando o sistema vai entrar em produção

DevOps

- Após aplicar tudo o que vimos sobre Desenvolvimento / Engenharia de Software, o sistema — ou um incremento dele, resultante de um sprint — está pronto para entrar em produção.
- Essa tarefa é conhecida pelos nomes de **implantação (deploy)**, **liberação (release)** ou **entrega (delivery)** do sistema.
- Independentemente do nome, ela não é tão simples e rápida como pode parecer.



DevOps

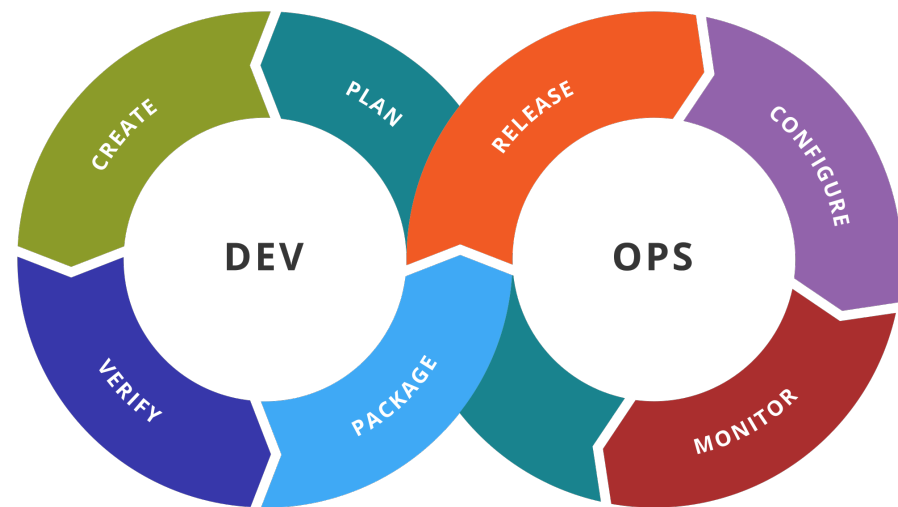
- Historicamente, em organizações tradicionais, a área de Tecnologia da Informação era dividida em dois departamentos:
 - **Departamento de Sistemas** (ou Desenvolvimento), formado por desenvolvedores, programadores, analistas, arquitetos, etc.
 - **Departamento de Suporte** (ou Operações), no qual ficavam alocados os administradores de rede, administradores de bancos de dados, técnicos de suporte, técnicos de infraestrutura, etc.
- Hoje em dia, é fácil imaginar os problemas causados por essa divisão.

DevOps

- Na maioria das vezes, a área de suporte tomava conhecimento de um sistema **na véspera** da sua implantação.
- Consequentemente, a implantação **poderia atrasar por meses**, devido a uma variedade de problemas que não tinham sido identificados.
- Dentre eles, podemos citar a **falta de hardware** para executar o novo sistema ou a nova funcionalidade, **problemas de desempenho**, **incompatibilidades** com o banco de dados de produção, vulnerabilidades de **segurança**, etc.
- No limite, esses problemas poderiam resultar no **cancelamento da implantação** e no abandono do sistema.
- Nesse modelo tradicional, existia um stakeholder importante — os administradores de sistemas — que tomava conhecimento das características e **requisitos não-funcionais** de um novo software na véspera da implantação.

DevOps

- Então, para facilitar a implantação e entrega de sistemas, foi proposto o conceito de **DevOps**.
- Por ser um termo recente, ele ainda não possui uma definição consolidada.
- Mas seus proponentes gostam de descrever DevOps como *um movimento que visa unificar as culturas de desenvolvimento (Dev) e de operação (Ops), visando permitir a implantação mais rápida e ágil de um sistema.*



“

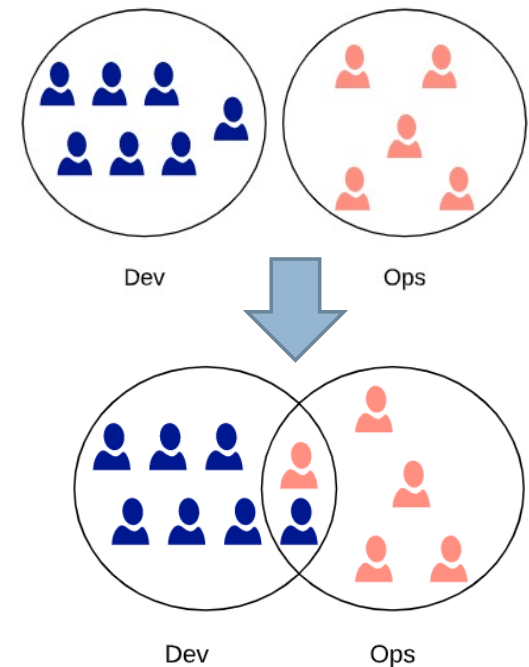
“Em vez de iniciar as implantações à meia-noite de sexta-feira e passar o fim de semana trabalhando para concluí-las, as implantações ocorrem em qualquer dia útil, quando todos estão na empresa e sem que os clientes percebam — exceto quando encontram novas funcionalidades e correções de bugs.”

The DevOps Handbook - 2016

Gene Kim, Jez Humble, Patrick Debois e John Willes

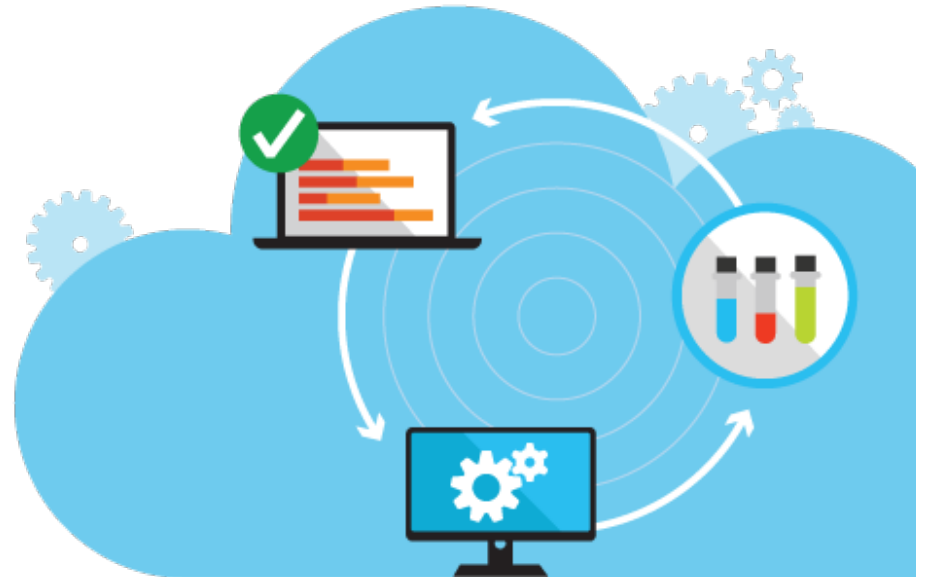
DevOps

- No entanto, DevOps **não** advoga a criação de um **profissional novo**, que fique responsável tanto pelo desenvolvimento como pela implantação de sistemas.
- Em vez disso, defende-se uma **aproximação** entre o pessoal de **desenvolvimento** e o pessoal de **operações** e vice-versa, visando fazer com que a implantação de sistemas seja mais ágil e menos traumática.
- A ideia é evitar dois grupos independentes: desenvolvedores e operadores, com pouca ou nenhuma interação entre eles.
- Em vez disso, defende-se que esses profissionais atuem em conjunto desde os primeiros sprints de um projeto.



DevOps

- DevOps advoga a automatização de todos os passos necessários para colocar um sistema em produção e monitorar o seu correto funcionamento.
- Isso requer a adoção de práticas como testes automatizados.
- Mas também requer o emprego de novas práticas e ferramentas, tais como **Integração Contínua** (Continuous Integration) e **Deployment Contínuo** (Continuous Deployment).



DevOps - Mundo Real

- O termo DevOps começou a ser usado no final dos anos 2000 por profissionais **frustrados** com os atritos constantes entre as equipes de desenvolvimento e de operações.
- Então, eles convenceram-se de que uma solução seria a adoção de princípios ágeis não apenas na fase de **desenvolvimento**, mas também na fase de **implantação** de sistemas.

- Mas também requer o emprego de novas práticas e ferramentas, tais como **Integração Contínua** (Continuous Integration) e **Deployment Contínuo** (Continuous Deployment).



Princípios para entrega de software

- **Crie um processo repetível e confiável para entrega de software.**
 - A entrega de software não pode ser um evento traumático, com passos manuais e sujeitos a surpresas.
- **Automatize tudo que for possível.**
 - Todos os passos para entrega de um software devem ser automáticos, incluindo seu build, a execução dos testes, a configuração e ativação dos servidores e da rede, a carga do banco de dados, etc.
- **Mantenha tudo em um sistema de controle de versões.**
 - T-U-D-O refere-se não apenas a todo o código fonte, mas também arquivos e scripts de administração do sistema, documentação, páginas Web, arquivos de dados, etc.

Princípios para entrega de software

➤ **Se um passo causa dor, execute-o com mais frequência e o quanto antes.**

- A ideia é antecipar os problemas, antes que eles se acumulem e as soluções fiquem complicadas.
- O exemplo clássico é o de Integração Contínua. Se um desenvolvedor passa muito tempo trabalhando de forma isolada, ele e o seu time podem depois ter uma grande dor de cabeça para integrar o código.

➤ **“Concluído” significa pronto para entrega.**

- “Concluído”, em projetos de software, deve ter uma semântica clara, isto é: 100% pronto para entrar em produção.

➤ **Todos são responsáveis pela entrega do software.**

- Não admite-se mais que os times de desenvolvimento e de operações trabalhem em silos independentes e troquem informações apenas na véspera de uma implantação.